

INFORME TÉCNICO — TALLER SEMANA 9
CRM "Gestión Integral de Negocios" — ALP-365

Docente: Ing. José Daniel Cadenas L. Fecha: 30/09/2026

 Datos del estudiante

Nombre y apellido: Fabian Ortiz

Cédula: V31366153 (IMPORTANTE: con esta cédula se asignan las preguntas de la defensa)

Sección: B

Grupo N°: _____

Correo institucional: ortiz331fabian@gmail.com

Repositorio GitHub: <https://github.com/7FABIAN/Taller-Laravel9>

 Asignación de preguntas de defensa

Los 3 últimos dígitos de mi cédula son: 1 5 3

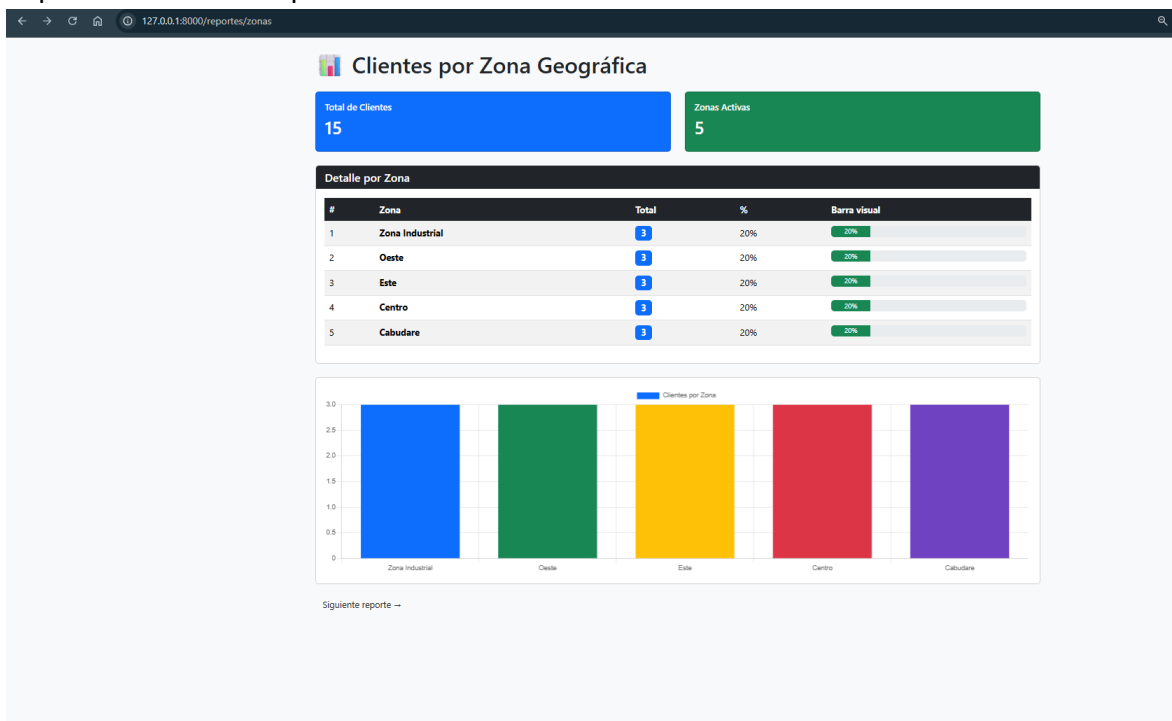
La suma de estos dígitos es: 9

El resultado de (suma % 10) es: 9

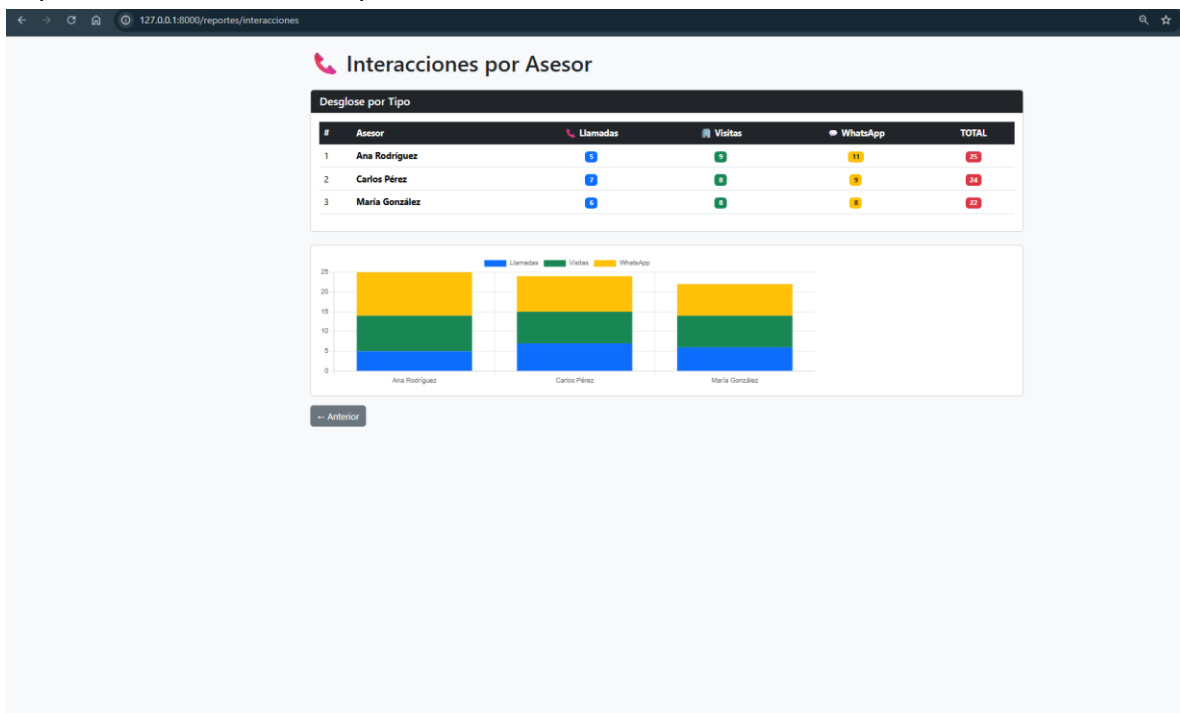
Mi SET de preguntas asignado es: SET J

 EVIDENCIA 1 — Capturas de los 2 reportes (2%)

Reporte 1 — Clientes por Zona:



Reporte 2 — Interacciones por Asesor:



✓ EVIDENCIA 2 — Respuestas a las 6 preguntas de comprensión (3%)

P1: Si cambio WHERE zona_geografica = 'Oeste' por WHERE zona_geografica LIKE '%Este%', ¿qué clientes aparecerán?

R1: Aparecen tanto los clientes con zona geografica "este" como "oeste"

P2: ¿Qué pasaría si quito el GROUP BY y dejo solo SELECT zona_geografica, COUNT(*)? ¿Funcionaría? ¿Por qué?

R2: Sin "group by" generaria un error ya que zona_geografica es una sola columna, y count * lo que hace es comprimir filas.

P3: Tenemos 3 asesores: Ana (5 clientes), Carlos (0 clientes), María (3 clientes). ¿Qué mostrará un INNER JOIN para Carlos? ¿Y un LEFT JOIN?

R3: un "inner join" solo devuelve coincidencias con lo cual carlos será excluido del resultado y un "left join" conserva todos los registros de la tabla de la izquierda, independientemente de si tienen o no correspondencia en la tabla de la derecha.

P4: En el reporte de interacciones por asesor, ¿por qué usamos LEFT JOIN y no INNER JOIN?

R4: un left join muestra a Ana, María y también a Carlos (quien tiene 0 clientes o interacciones) y un inner join eliminaría automáticamente a Carlos del reporte porque no hay coincidencias en la tabla de interacciones.

P5: ¿Qué hace DB::raw() y por qué debemos usarlo con cuidado?

R5: Si metes datos escritos por usuarios ahí sin protección, dejas tu app vulnerable a ataques de Inyección sql.

P6: ¿Qué diferencia hay entre ->get() y ->first()?

R6: el get devuelve una lista con todos los registros que encuentre y first solo muestra el primer registro que encuentre.

✓ EVIDENCIA 3 — Código comentado de UN método (2%)

```
public function clientesPorZona()
{
    // 1. Consulta: parte de la tabla clients
    $zonas = DB::table('clients')
        ->select(
            'zona_geografica', // la columna por la que se agrupa
            DB::raw('COUNT(*) as total') // cuenta los clientes de cada zona)
        ->groupBy('zona_geografica') // una fila por zona
        ->orderByDesc('total') // la zona con más clientes primero
        ->get(); // ejecuta y devuelve una collection
    // 2. Suma la columna total de todas las zonas
    $totalGeneral = $zonas->sum('total');
    // 3. Recorre cada zona y le agrega su porcentaje
    $zonasConPorcentaje = $zonas->map(function ($zona) use ($totalGeneral) {
        $zona->porcentaje = $totalGeneral > 0
            ? round(($zona->total / $totalGeneral) * 100, 2) // regla de tres con 2 decimales
            : 0; // evita dividir entre cero
        return $zona; //necesita devolver el elemento
    });
    // 4. Extrae las columnas para el gráfico de Chart.js
    $labels = $zonasConPorcentaje->pluck('zona_geografica')->toArray(); // nombres de
zonas
    $data = $zonasConPorcentaje->pluck('total')->toArray(); // cantidades
    // 5. Envía las variables a la vista
```

```
    return view('reports.zonas', compact('zonasConPorcentaje', 'totalGeneral', 'labels',  
'data'));  
}
```

✅ EVIDENCIA 4 — Retos de modificación resueltos (2%) Reto 1 (filtro > 15

// aqui uso el filter de la pista para el reto 1 del 15%

```
$zonasConPorcentaje = $zonasConPorcentaje->filter(function ($zona) {  
    return $zona->porcentaje > 15;  
});
```

Reto 2 (desglose por día): NO LO RESOLVI

✅ EVIDENCIA 5 — Reflexión breve (100 palabras)

¿Qué concepto fue más difícil?

La verdad lo que me pareció más difícil fue solucionar esos pequeños errores por letras que estaban en mayúsculas o signos mal puestos (como un punto, una coma o una barra donde no iba), porque uno revisa y recorre todo el código y aun así a veces cuesta muchísimo encontrar dónde está el detalle que hace que no compile el código y te de algun error

¿Cómo aplicarás esto al CRM final?

Eso lo aplicaria en el crm final para estar mas pendiente de la sintaxis y ser mas cuidadoso con los nombres que uno le pone a las tablas y a los campos porque un simple descuido con una letra te quita mucho tiempo

✅ EVIDENCIA 6 — Commit en GitHub [CAPTURA DEL COMMIT]